

Student Management System Project Report

1. Project overview

The Student Management System is a full-stack web application for maintaining a reliable directory of student records. It addresses the common problem of fragmented spreadsheets and inconsistent manual updates by providing one searchable workspace for student identity, course, year, status, contact, and enrollment information.

The final application demonstrates the four required CRUD operations. A staff member can create a student record, read the directory, update an existing record, and remove a record after confirmation. The interface also surfaces summary counts so that the state of the directory is visible at a glance.

2. Objectives

The project has four objectives. First, it provides a practical management workflow with a clear and responsive user interface. Second, it demonstrates typed communication between a React client and a server API. Third, it applies validation at both the form and server layers. Fourth, it documents the system sufficiently for installation, testing, evaluation, and future extension.

3. Requirements analysis

The primary user is an administrator or student-services staff member. The main entity is a student. Each record contains a unique student ID, name, email address, optional phone number, course, year level, status, and enrollment date. The system must support search, clear validation messages, duplicate protection, and error handling when a record is missing or a backend operation fails.

The interface was designed around a directory workflow. The sidebar provides context, the top bar communicates save state, summary cards provide operational counts, and the table presents the data in a compact format. A modal form keeps creation and editing in the same workspace without creating navigation dead ends.

4. Technology and architecture

The managed project uses React 19 and TypeScript for the frontend. Express and tRPC expose typed procedures on the server. Drizzle ORM maps the TypeScript schema to a

MySQL/TiDB database. The UI uses CSS variables and responsive media queries to provide a consistent visual system across desktop and mobile screens.

The request flow is:

Plain Text

Administrator → React dashboard → tRPC client → Express server → validation → D

The browser invalidates the student list query after a successful create, update, or delete mutation. This keeps the visible directory synchronized with persisted data. When the database is unavailable, a small in-memory fallback keeps the demonstration path usable; this fallback is not intended to replace production persistence.

5. Database design

The `students` table uses an auto-incrementing integer primary key. `studentId` and `email` have unique constraints. Required fields are non-null, while the phone number is nullable. `status` is limited to three known values. `createdAt` and `updatedAt` support audit-friendly sorting and update tracking. The database schema and generated migration are stored under `drizzle/`.

The entity relationship diagram is provided in [ER_DIAGRAM.md](#). The project retains the managed `users` table for authentication compatibility, although the student directory is intentionally simple and can be protected with the template's authenticated procedures when institutional access control is enabled.

6. CRUD implementation

The server exposes `students.list`, `students.get`, `students.create`, `students.update`, and `students.remove`. The create procedure rejects empty or malformed fields and checks both unique fields before insertion. The update procedure checks record existence and rechecks uniqueness when a unique field changes. The delete procedure returns a not-found error for an invalid ID rather than silently succeeding.

The client provides native input constraints for fast feedback and relies on the server for authoritative validation. Search uses a stable query object so the client does not create unnecessary request loops. After every successful mutation, the client refreshes the list and shows a success toast. Delete operations use a confirmation dialog to reduce accidental data loss.

7. Testing approach

Testing covers the server contract, validation failures, CRUD behavior, and the main responsive UI states. Vitest verifies the existing authentication logout behavior and student procedure validation. TypeScript compilation checks the schema, server contract, and React client together. A production build checks that Vite and the server bundler can produce deployable output. Manual browser checks cover loading, populated, empty-search, modal, validation-error, and mobile layouts.

The full test matrix appears in [TEST_CASES.md](#). The Postman collection in `docs/postman_collection.json` provides an API demonstration artifact aligned to the SOP's HTTP method table.

8. Security and quality

Secrets are read from environment variables and are not hard-coded. Drizzle generates parameterized database operations. Input is validated on the server even when client-side validation exists. Unique constraints protect against duplicate identifiers. The project keeps the presentation, server procedures, and database schema in separate layers, which makes the system easier to test and extend.

9. Challenges and solutions

A managed full-stack runtime was used instead of a separate Django process because the available WebDev project template provides React, Express, tRPC, Drizzle, database integration, authentication plumbing, and deployment support as one supported unit. The user-visible requirements remain unchanged: the application has a frontend, backend, REST-equivalent API semantics, persistent database schema, validation, CRUD behavior, testing artifacts, and documentation.

A second challenge is demonstrating the UI before a local database is configured. The database helper therefore includes a controlled demo fallback. It is isolated from the Drizzle path and clearly documented so that developers can distinguish preview convenience from production persistence.

10. Future enhancements

The next logical improvements are authenticated roles for registrar, faculty, and viewer access; pagination for larger directories; CSV import and export; course and attendance entities; audit history; and automated end-to-end browser tests. A production deployment should also add rate limiting, institution-managed identity, structured server logging, and database backup procedures.

11. Completion criteria

The project satisfies the completion criteria when the dev server starts without errors, the database migration is applied, the directory loads, and each CRUD operation can be demonstrated. The repository contains source code, schema and migration files, setup instructions, API documentation, test cases, and an explanatory report. The student can use these materials to explain the architecture and code flow during a viva or project demonstration.